

Summary of Changes

P- 20

Networking Team

Peer Team Feedback:

After receiving feedback from other teams during the integration review, two key features were identified as missing from our current system: a **Turn Timer** and a **Content Filter** System. Both were essential to improving fairness, gameplay flow, and creating a safer environment for players. We've now implemented both features and ensured they integrate cleanly into our existing system architecture.

1. Turn Timer Functionality

The feedback pointed out the lack of a mechanism to handle players who go idle or take too long to make a move. To address this, we implemented a TurnTimer class that monitors each player's turn duration. The timer starts at the beginning of a player's turn and counts down. If the player fails to act within the allowed time, the system flags a timeout and simulates disconnection from the game session. In addition to managing timeout scenarios, the timer includes the ability to:

- Pause during unexpected issues like disconnection.
- Resume when the player reconnects.
- Reset upon successful turn completion.
- Notify the player when only 10 seconds remain on the clock.

This change improves the pacing of the game and prevents stalling, which is especially important in a real-time or competitive setting.

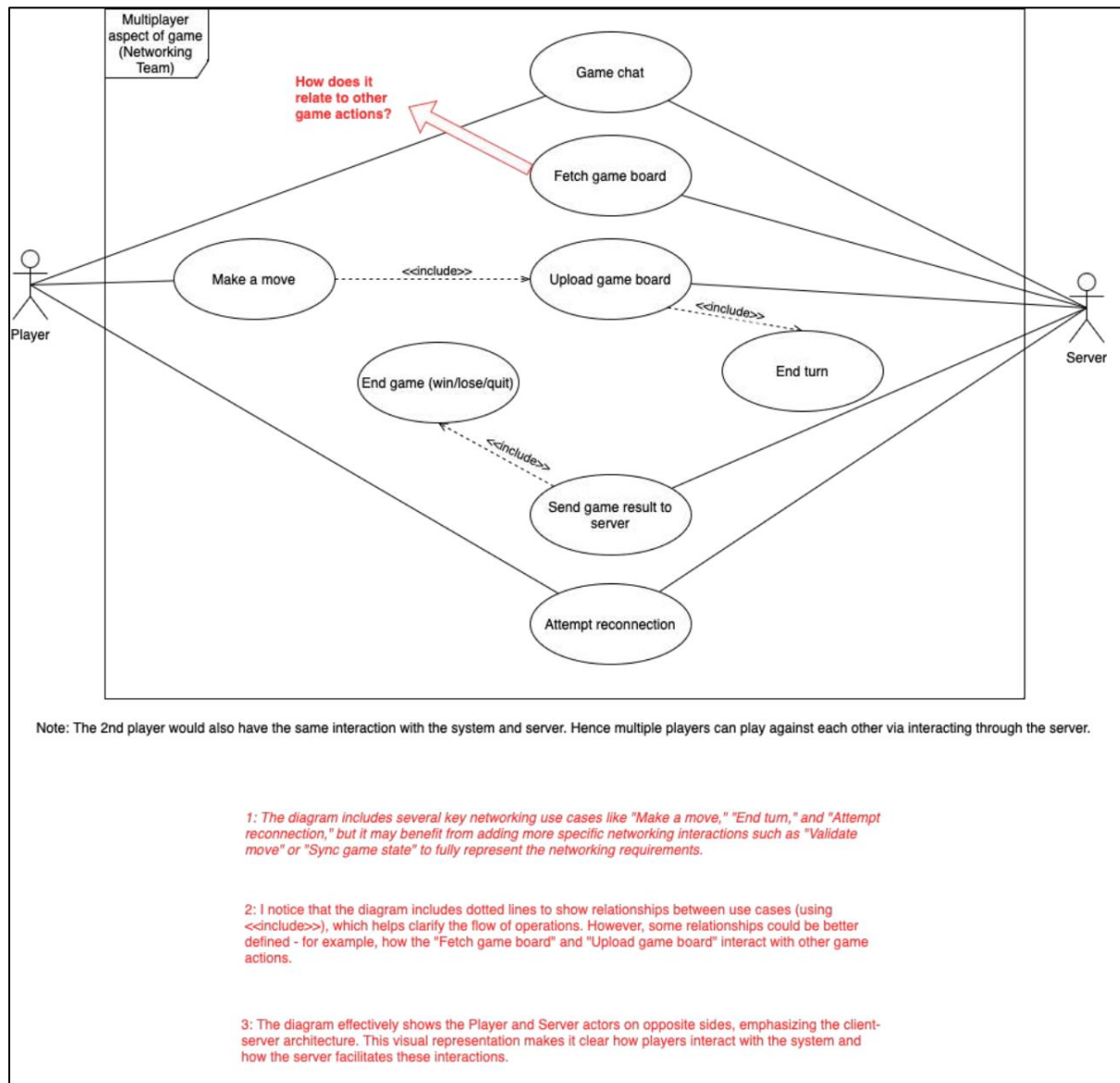
2. Content Filter

Another suggestion was to handle inappropriate language in the in-game chat. As a solution, we extended the ChatManager to include a content filter that checks all incoming messages against a curated list of offensive words. If a message contains any of these terms, it is automatically rejected and not added to the chat history. The player is warned via a console message. This ensures the chat system promotes respectful communication and discourages toxic behavior. The list of inappropriate words was sourced from an open-source project and adapted to cover a wide range of offensive terms. Although reporting functionality is not yet fully implemented,

the system can easily be extended to flag users or notify moderators when inappropriate messages are blocked.

Use Case Diagram:

Following is the annotated diagram received from Iteration 2 feedback:

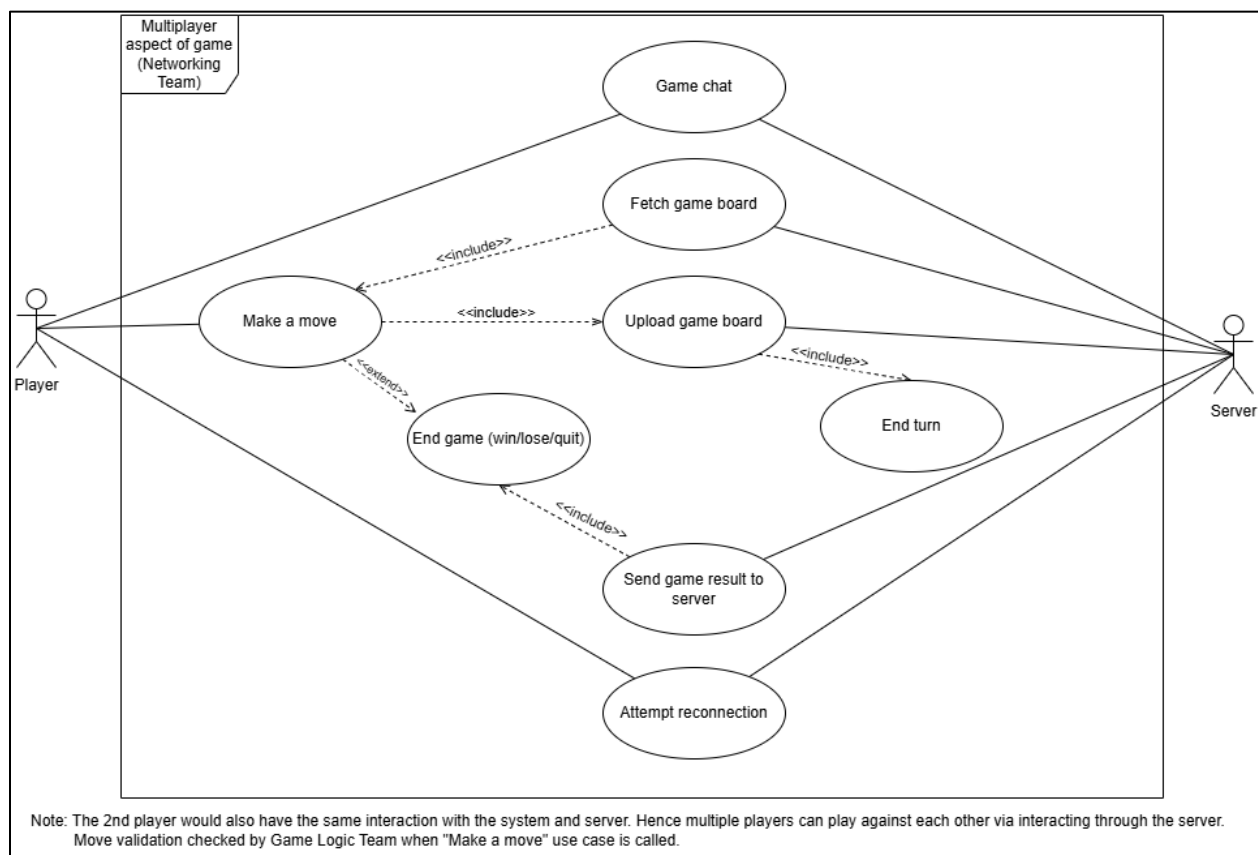


Updated Diagram:

Addressing feedback comment 1: In our project, move validation is done by Game Logic Team and occurs when “Make a move” use case is called (added as a note in updated diagram). “Sync game state” is performed by “Fetch game board” and “Upload game board” use cases.

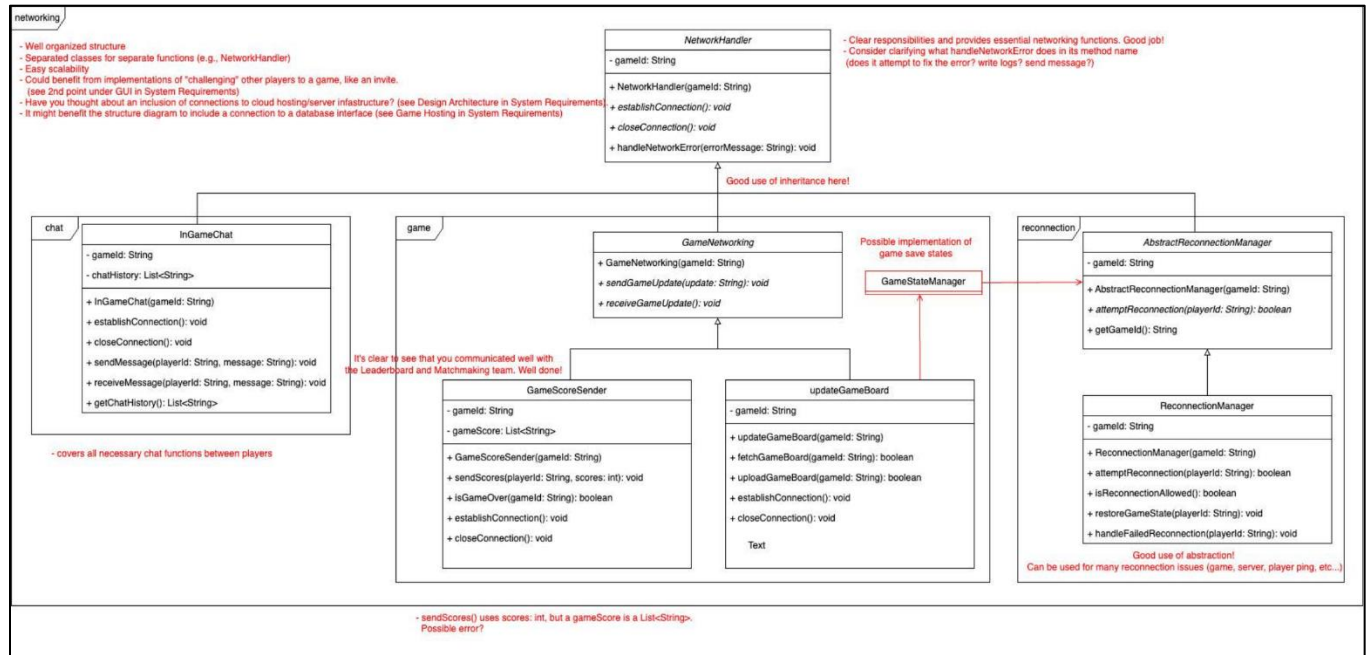
Addressing feedback comment 2: Added an <<include>> relation from “Fetch game board” to “Make a move” to show that a move is only made after updated game board is fetched from server. Similarly, “upload game board” occurs after a move is made and then the turn ends.

Addressing feedback from the other (2nd) team: Added an extend relation from “Make a move” to “End game”, so that when a player does not make a move, the game automatically ends.



Structure Diagram:

Following is the annotated diagram received from Iteration 2 feedback:



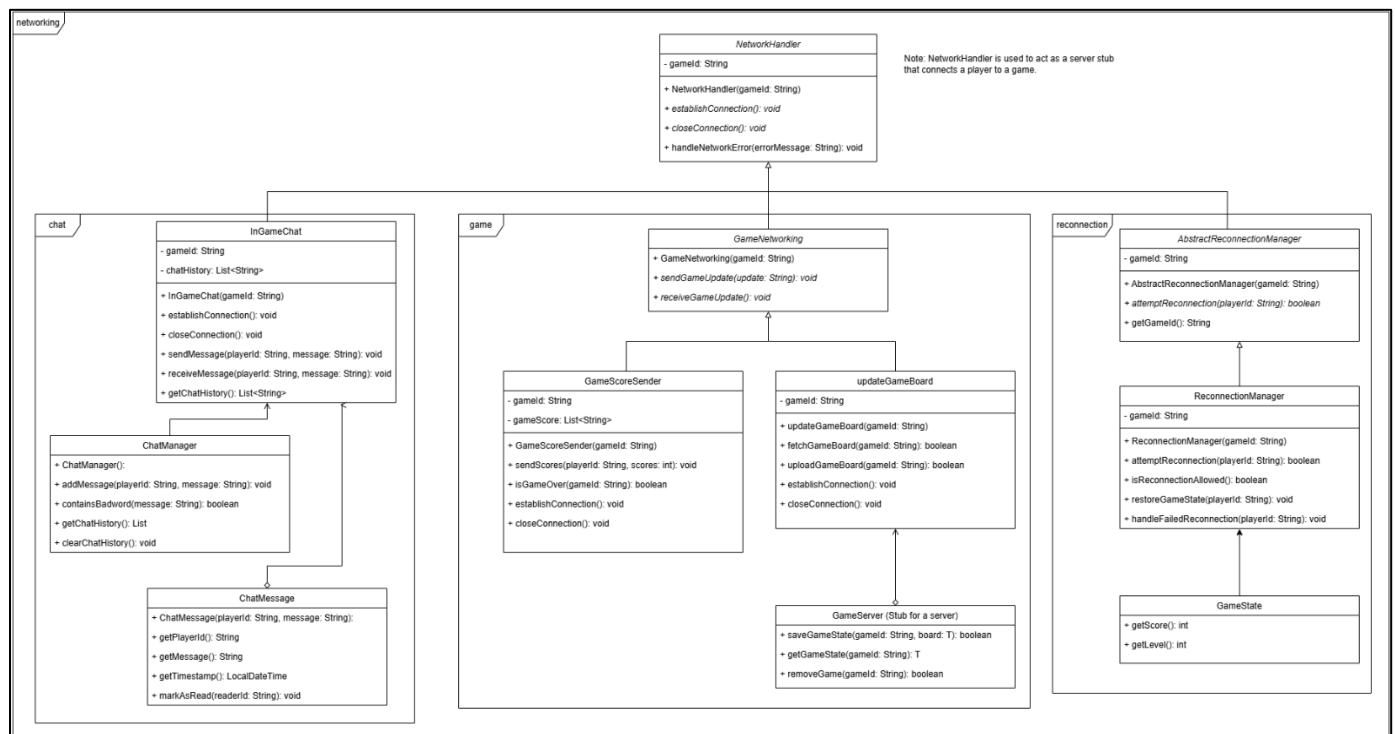
Updated Diagram:

First of all, new classes have been added that weren't created during iteration 2.

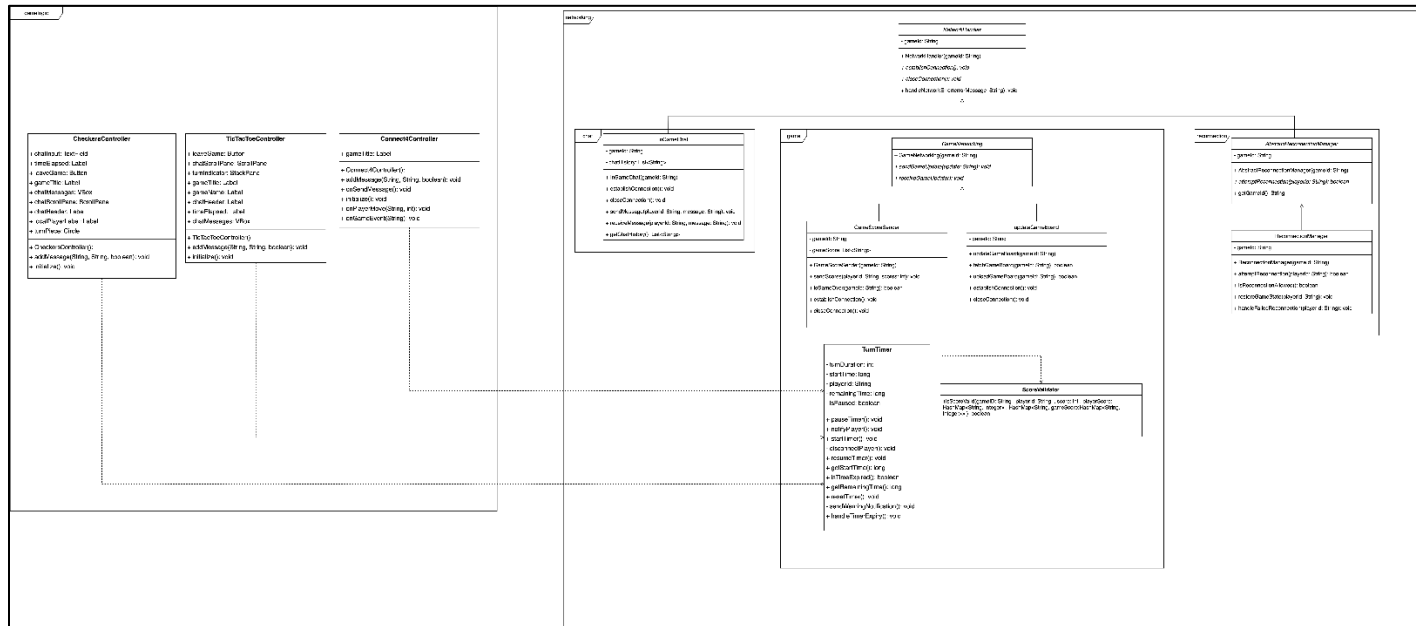
As highlighted in the feedback and project requirements, a game server class has been added which acts as a stub.

A GameState class has been added in the reconnection package as per the feedback.

A note has been added which clarifies what NetworkHandler function is.

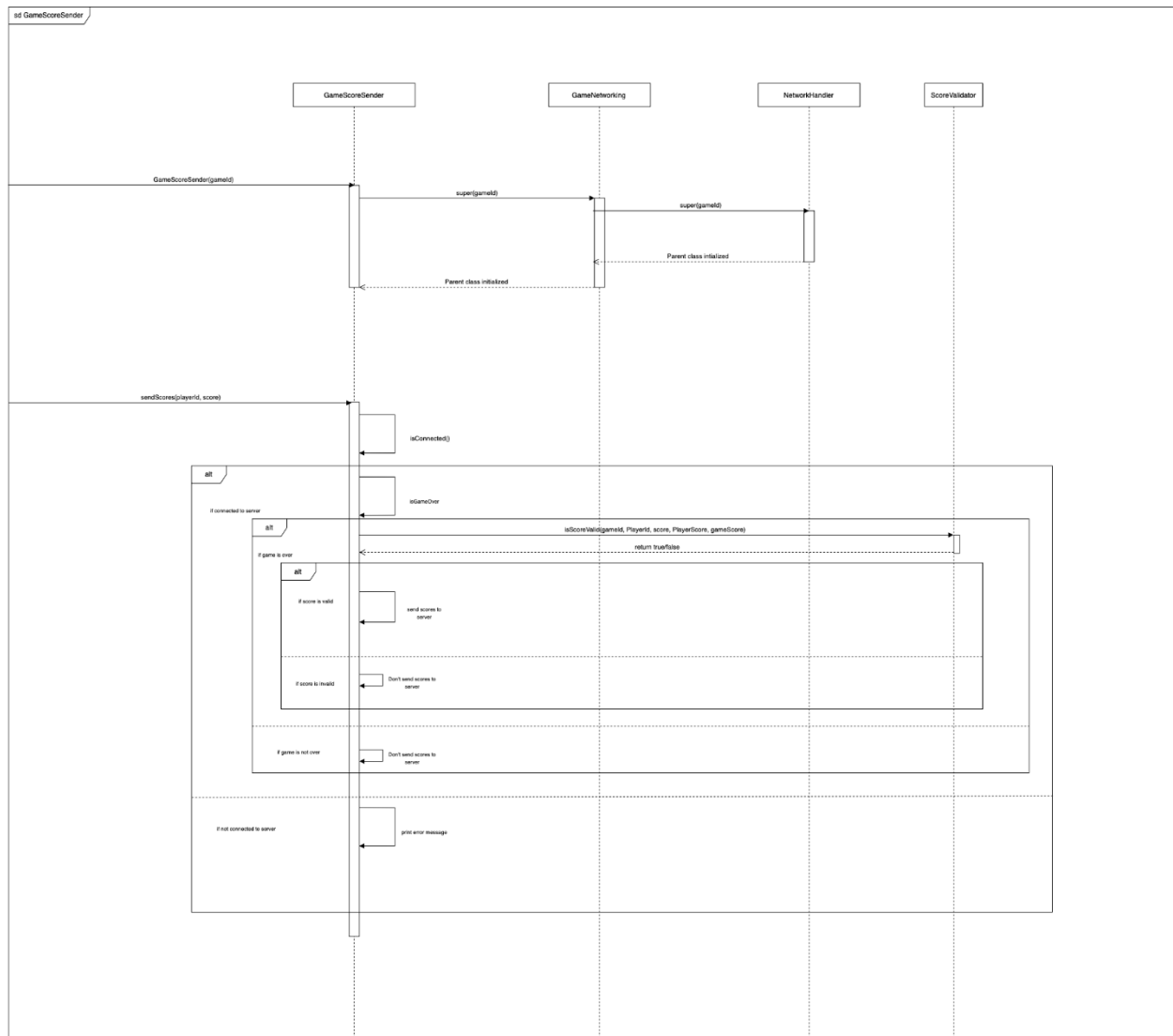


Another version of the updated structure diagram is the following which focuses on the newly added Turn Timer functionality:



Sequence Diagram:

Following is a new sequence diagram created:



Game Logic Team

In the iteration 2 , although no new feature requests were there for the Game Logic module by the other groups, significant improvements were implemented internally to enhance functionality and maintainability. A key change was made to the CheckersMove class by introducing a static nested Move class. This new structure allows each move to store its destination row and column along with a list of captured positions, which was not possible with the previous logic. The update supports both single and multi-jump moves by encapsulating captured piece coordinates in a structured way. This change improves how the game handles complex move sequences and paves the way for future features like move replay or undo functionality.

```
public class CheckersMove { 8 usages  ± 30212344 +2

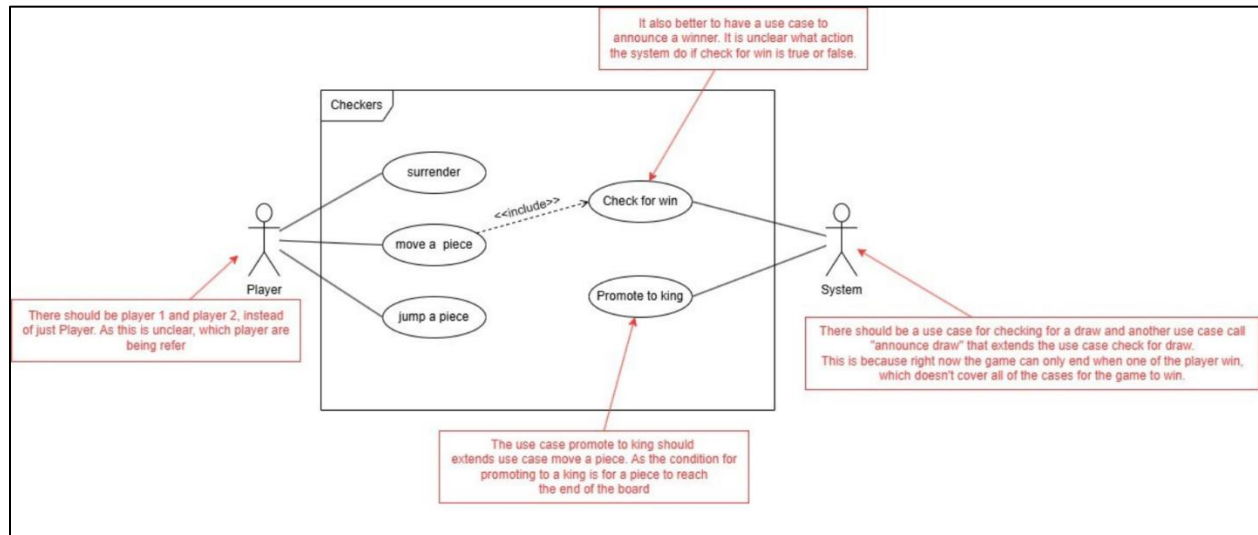
    public static class Move { 25 usages  ± 30212344
        public int destRow, destCol; 9 usages
        public List<int[]> capturedPositions; 4 usages

        public Move(int destRow, int destCol, List<int[]> capturedPositions) {
            this.destRow = destRow;
            this.destCol = destCol;
            this.capturedPositions = capturedPositions;
        }
    }
}
```

Additionally, the planning document was revised to reflect the full scope of the project rather than just Iteration 1. The original version was limited and lacked alignment with the complete development cycle. The updated version includes detailed timelines, responsibilities, and goals across all project phases, ensuring that the planning reflects actual progress and deliverables.

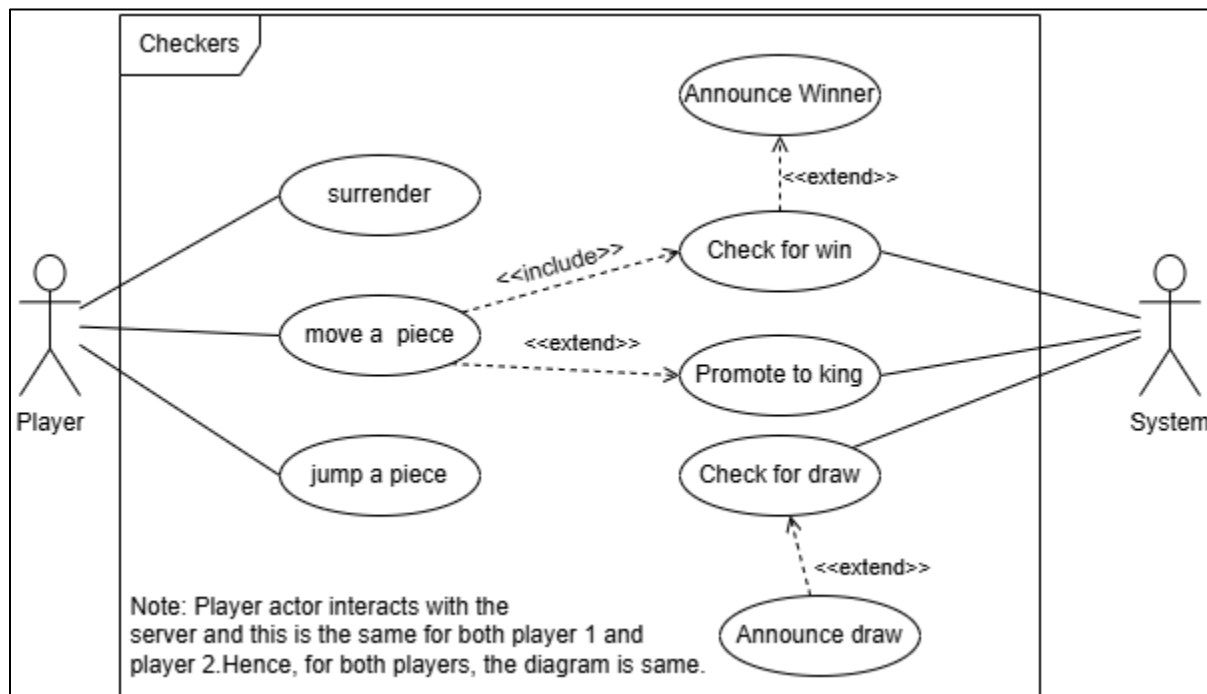
Use Case Diagrams:

Following is the annotated diagram received from Iteration 2 feedback:



Updated Diagram:

Added a note to make the player actor clear. Added use cases for announcing winner, check for draw, and announce draw. Created a new extend relation from "move a piece" to "promote to king".



Similarly, use case diagrams for tictactoe and connect4 are also updated.

Authentication Team

As part of the integration process and based on suggestions from other teams, the Authentication group made several important changes to improve functionality, maintainability, and interoperability. One of the key changes was shifting from in-memory data storage to a persistent CSV-based system (userdata.csv). This allows for cleaner and more consistent handling of user data across modules and ensures that information is stored even after the program closes.

Additionally, the password reset workflow was redesigned for better security. Now, when users forget their password, they must first verify their email address. After that, a reset token is generated and displayed on the screen, which the user must enter correctly before being allowed to set a new password.

We also updated the "Change Password" functionality within the homepage. Previously, there was no verification process, but now the system first verifies the registered email and then asks for the old password before allowing the user to set a new one. Another significant update involved the login system, originally, users logged in using their email address, but now they use their username, which aligns more closely with standard user expectations and makes the interface more intuitive. These improvements not only enhance user security but also improve cross-team compatibility, especially with the Leaderboard and Matchmaking systems, which rely on user ID and username consistency. Diagrammatically, we revised the login and password reset flows to reflect these new steps, making the logic easier to follow and debug.

```
1 package Authentication;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 public class UserDatabase {
7     // Instead of directly storing User objects, we store field-by-field data.
8     private static final Map<Integer, Map<String, Object>> userData = new HashMap<>();
9     // For quick lookup by email, store email -> userID
10    private static final Map<String, Integer> emailIndex = new HashMap<>();
11    private static int nextId = 1;
12}
```

```

package Authentication;

import ...

/**
 * The UserDatabase class handles storing, loading, and managing users from a CSV file called "userdata.csv"
 * This class uses an in-memory list of User objects and keeps it in sync with the CSV file.
 */

public class UserDatabase {
    private static final String FILE_PATH = "userdata.csv"; // Path to the CSV file
    private static List<User> users = new ArrayList<>();

    // Static block: loads users from CSV when class is first accessed
    static {
        loadUsersFromCSV();
    }

    /**

```

FORGOT PASSWORD

Reset your OMG access

Registered Email

VERIFY EMAIL

[Back to Login](#)

FORGOT PASSWORD

Reset your DMG access

Reset Token

New Password

Confirm New Password

RESET PASSWORD

[Back to Login](#)

Summary of Change - Matchmaking/Leaderboard

We introduced an enum for game types, which was not originally planned. Up until last week, the game type was specified using an array of numbers, which initially worked well. However, maintaining a consistent format across the codebase became increasingly difficult. To address this, we decided to implement an enum for gameType to improve readability and maintainability.

Before:

```
*/
public class Player {
    private final double[] winRatio = new double[3]; // 1: TicTacToe, 2: Connect4, 3: Checkers
    private final int[] wins = new int[3];
    private final int[] losses = new int[3];
    private int level;
    private int userID;
    private boolean spectate;
    private final Rank[] rank = new Rank[3];
    private final int[] gameSignal = new int[3];
    private final int[] mmr = new int[3];

    /**
     * Constructs a Player object
     * @param level rank level of player
     * @param userID userID of the player from database
     * @param spectate whether player is spectating
     * @param rank rank of the player
     * @param gameSignal which game the player wants to play
     */
    public Player(double winRatioForGame, int level, int userID, boolean spectate, Rank rank, int gameSignal) {
        this.level = level;
        this.userID = userID;
        this.spectate = spectate;
        for(int i = 0; i < 3; i++) {
            this.winRatio[i] = 0.0;
            this.wins[i] = 0;
            this.losses[i] = 0;
            this.rank[i] = new Rank();
            this.gameSignal[i] = 0;
            this.mmr[i] = 0;
        }
    }
}
```

Now:

```
package MatchmakingLeaderboard;

/**
 * Enum for different game types within the matchmaking leaderboard system and other places
 */
public enum GameType {
    TIC_TAC_TOE(1),
    CONNECT_FOUR(2),
    CHECKERS(3);

    private final int gameCode;

    GameType(int gameCode) {
        this.gameCode = gameCode;
    }

    public int getGameCode() {
        return this.gameCode;
    }
}
```

One of the reviewing team members recommended implementing an ELO rating system for player rankings. After careful consideration, we chose to implement a **Matchmaking Rating (MMR)** system instead. This decision was based on MMR's flexibility and its ability to account for individual performance metrics beyond simple win/loss outcomes.

Our implementation uses individual win ratios as a proxy for player skill and dynamically adjusts MMR based on:

- Match outcome (win/loss/draw)
- Relative performance between players
- Game type (TicTacToe, Connect4, or Checkers)

We also updated the planning documents, added a proper timeline to track and manage work, and included details about the classes implemented for the Matchmaking-Leaderboard system. A brief overview of the ranking system and its functionality was also provided to help ensure clarity and alignment within the team.

GUI Team

The updated GUI demonstrates substantial improvements over the initial mockups, resulting in a more intuitive and cohesive user experience. Based on the feedback received during the P1 and P2 reviews, the home page has been significantly enhanced. The originally high-level mockup has evolved into a refined design that incorporates critical suggestions, such as adding placeholder text to the search bar, which guides users more effectively. While the overall structure remains faithful to the original plans, this enhanced version addresses key concerns and questions raised by reviewers.

In terms of gameplay, the interface now maintains consistent design elements across all game types by dynamically loading the board. This approach not only preserves a uniform look and feel regardless of whether the player chooses a random opponent or a friend but also ensures that the fundamental play experience is preserved with minor UI enhancements. The addition of a dynamic design also allows for a more streamlined and engaging experience during each match.

The chat system has been improved by incorporating the move update feature and adding a robust filtering mechanism that flags inappropriate language and issues warnings. These enhancements were introduced based on late-stage feedback and serve to improve both communication clarity and community standards. Additionally, the login system has been refined to mimic real-world authentication processes, complete with comprehensive validation and error messages. The implementation of a “player not found” error message during searches further contributes to a more robust and user-friendly system.

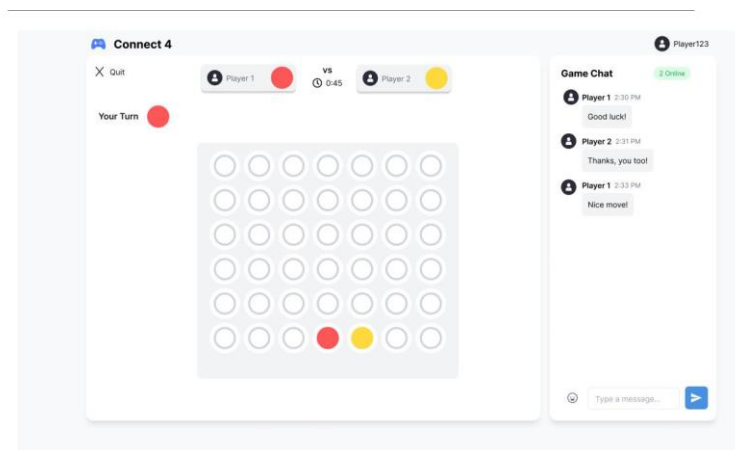
Although some suggestions were not incorporated, such as the addition of notifications for friend requests and a “show password” feature, these decisions were made after careful consideration. Friend requests now function automatically, eliminating the need for manual acceptance or

notifications, while the absence of a “show password” option aligns with privacy concerns and original project scope guidelines. Lastly, the introduction of loading screens to mimic network delays adds a realistic touch to the user experience, underlining the careful attention given to both usability and practical functionality.

Overall, these adjustments reflect a thoughtful integration of user feedback, balancing new features and necessary design constraints to achieve a polished, production-ready GUI that enhances both aesthetics and functionality.

As suggested by reviewer to stick to one design, we selected the below design

Mockup



Implementation – picture contrasting may look different from actual

